



University of Antwerp
| Faculty of Applied
Engineering

Lab 3 – Naming server

Distributed Systems

Andreas Gavrielides

University of Antwerp – imec, IDLab – Faculty of Applied Engineering, Belgium

Overview of lab sessions

- **Lab 1: DONE**

- Introductory lab & TCP/UDP
- Setup Remote nodes

- **Lab 2: DONE**

- REST protocol
- Remote nodes
- 2 sessions

- **Lab 3: Project -> Today**

- Naming server
- Remote nodes
- 3 sessions

- **Lab 4: Project**

- Discovery service
- Remote nodes
- 3 sessions

- **Lab 5: Project**

- Replication
- Remote nodes
- 4 sessions

- **Lab 6: Project**

- Agents
- Remote nodes
- 1 session

- **Lab 7: Project**

- GUI
- Remote nodes
- 1 session

- **Finishing project**

- Remote nodes
- 4 sessions

The Project -> System Y

- **Goal of the session:**

- To start working on the project
- To develop **Naming server** functionality

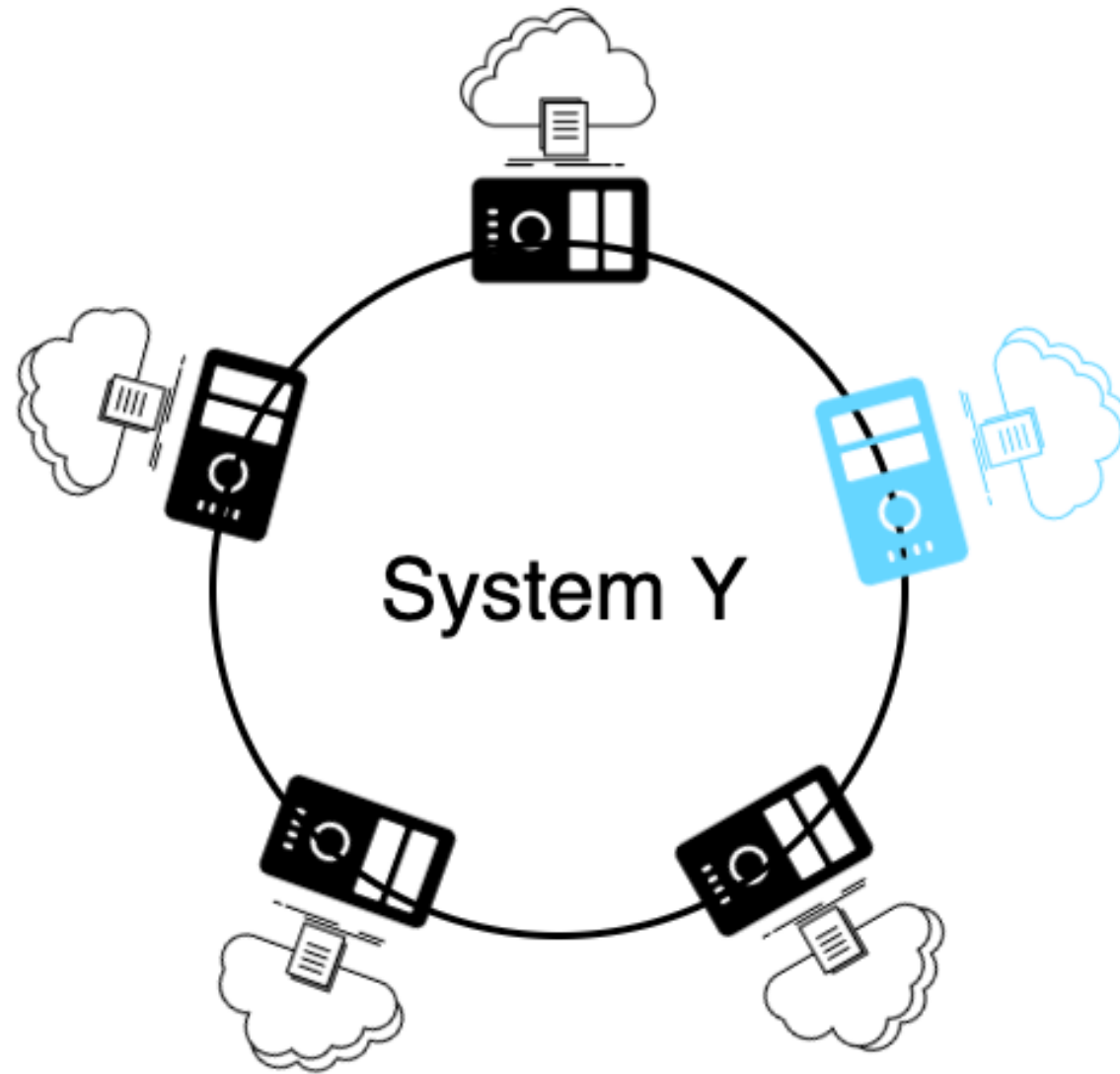
- **Team work:**

- Work in your groups (available on Blackboard), **divide the work!**

- **Project:**

- To develop a distributed file system called **System Y**, consisted of nodes that are organized in a **ring topology**.

Project topology



Naming server: specification 1

- **Naming server has a Map to store couples of (Integer, IP address)**
 - Integer is a positive value limited to 32768, as a **result of a hashing function**
 - IP address belongs to the unique node in your ring topology
- **Hashing function:**
 - The information about each node in the topology is recorded by Naming server, i.e., it keeps track of all nodes
 - Naming server performs hashing functionality
 - **Hashing function gets the name of the node as input, and returns its hash value as output**
- **The map should be saved on local disk (e.g., JSON file)**
- **The Naming server must be able to add and remove nodes from the Map**

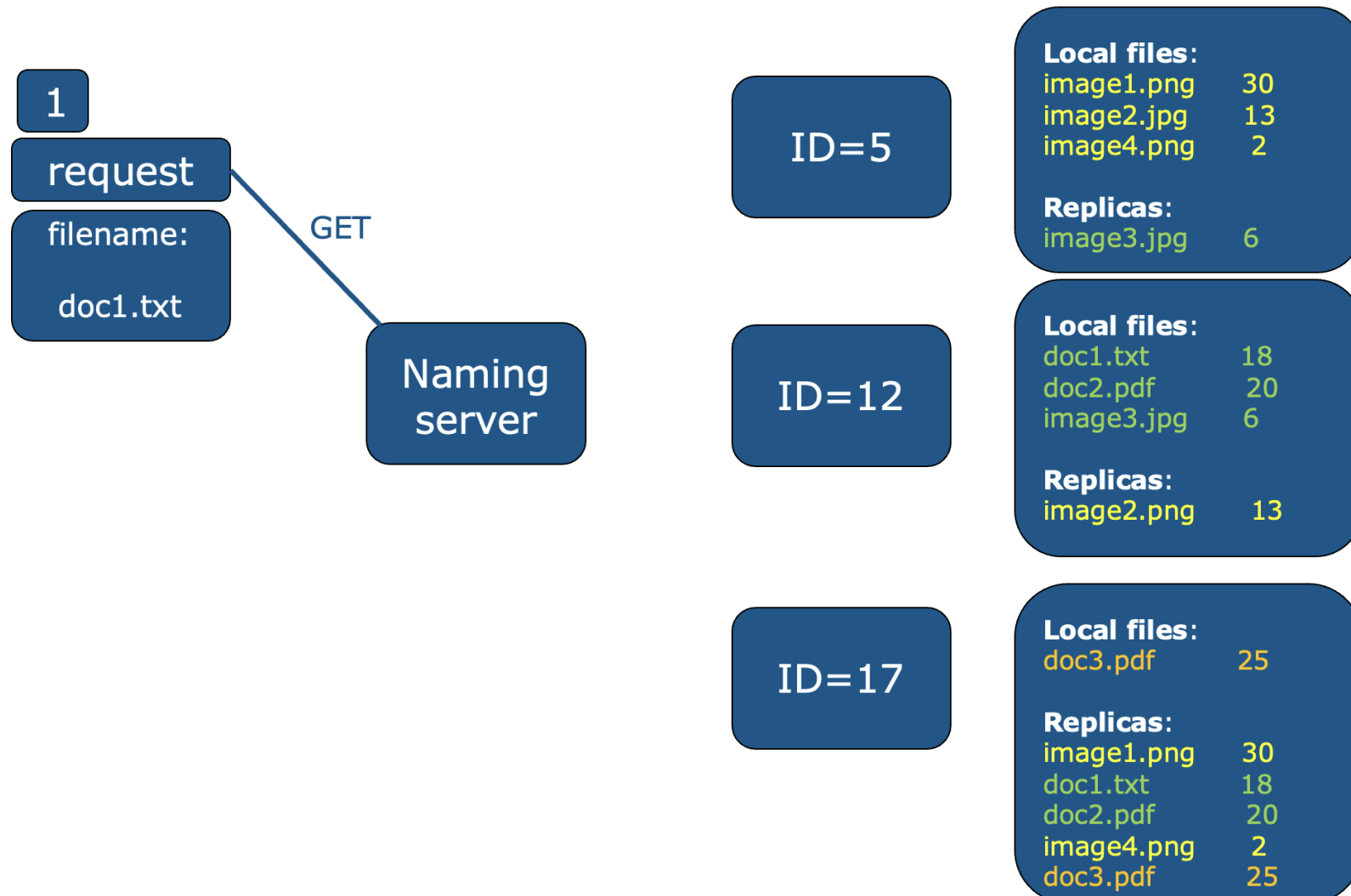
Naming server: specification 2

- Each node stores some files locally (e.g., .png, and .jpg images, .pdf files, .txt, etc.) – **owner of the file**
- Each node should be able to find out where is a specific file located (at which node)
 - This is possible via Naming server, as it is the only node that has this global information
- **Node(s) talks to Naming server using REST**
 - It sends the name of the file that it is looking for
 - Naming server calculates a hash value of the filename
 - Naming server determines the node ID at which a file with filename is stored
 - Naming server extracts the IP address of this node from the Map and sends it back to the node that sent the request

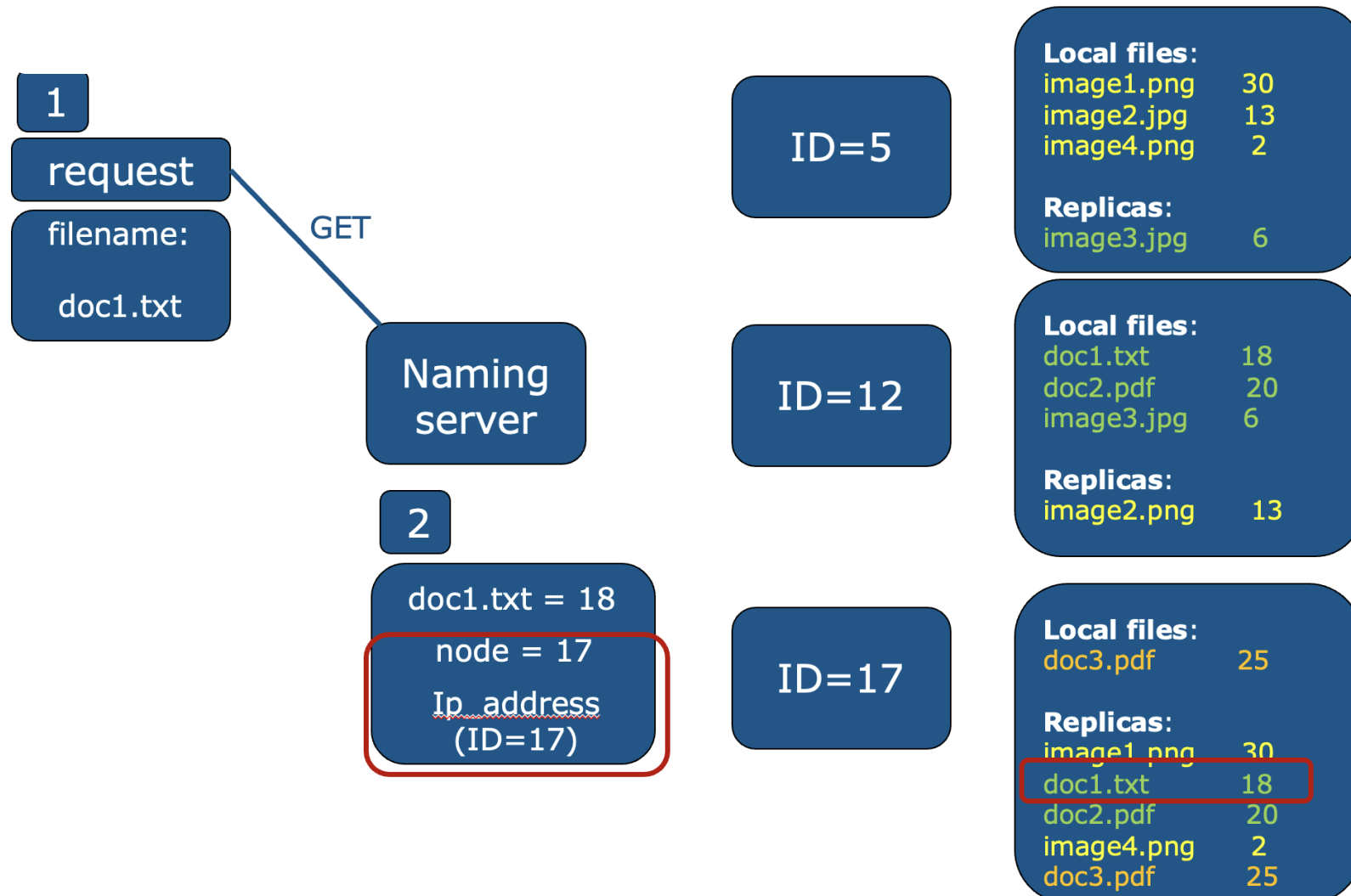
Naming server: specification 2

- **Algorithm that Naming server uses to determine at which node is a file stored:**
 - Suppose N is the collection of nodes with a hash smaller than the hash of the filename. Then the node with the smallest difference between its hash and the file hash is the owner of the file. If N is empty, the node with the biggest hash stores the requested file.

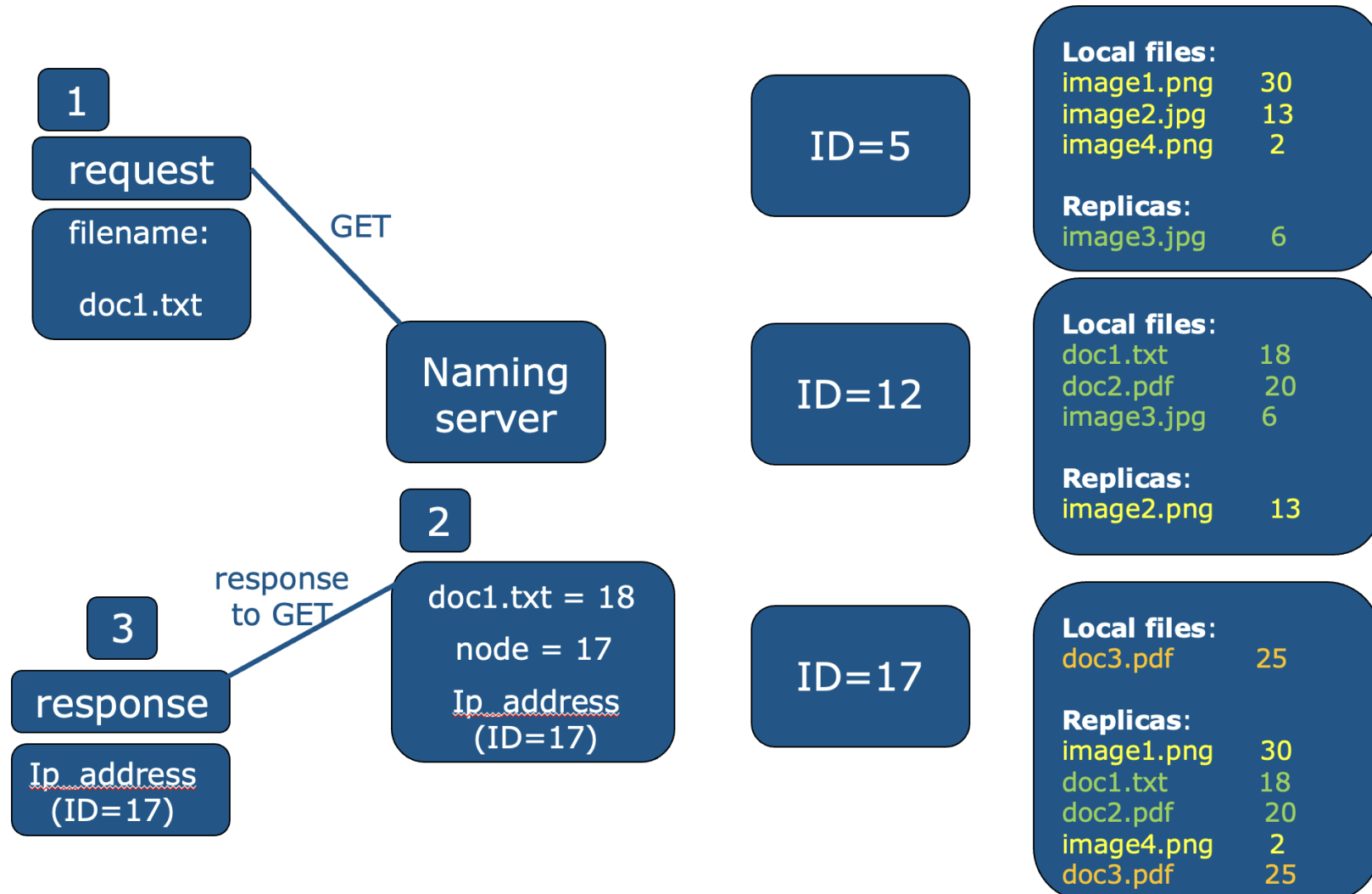
Naming server: specification 2, Example



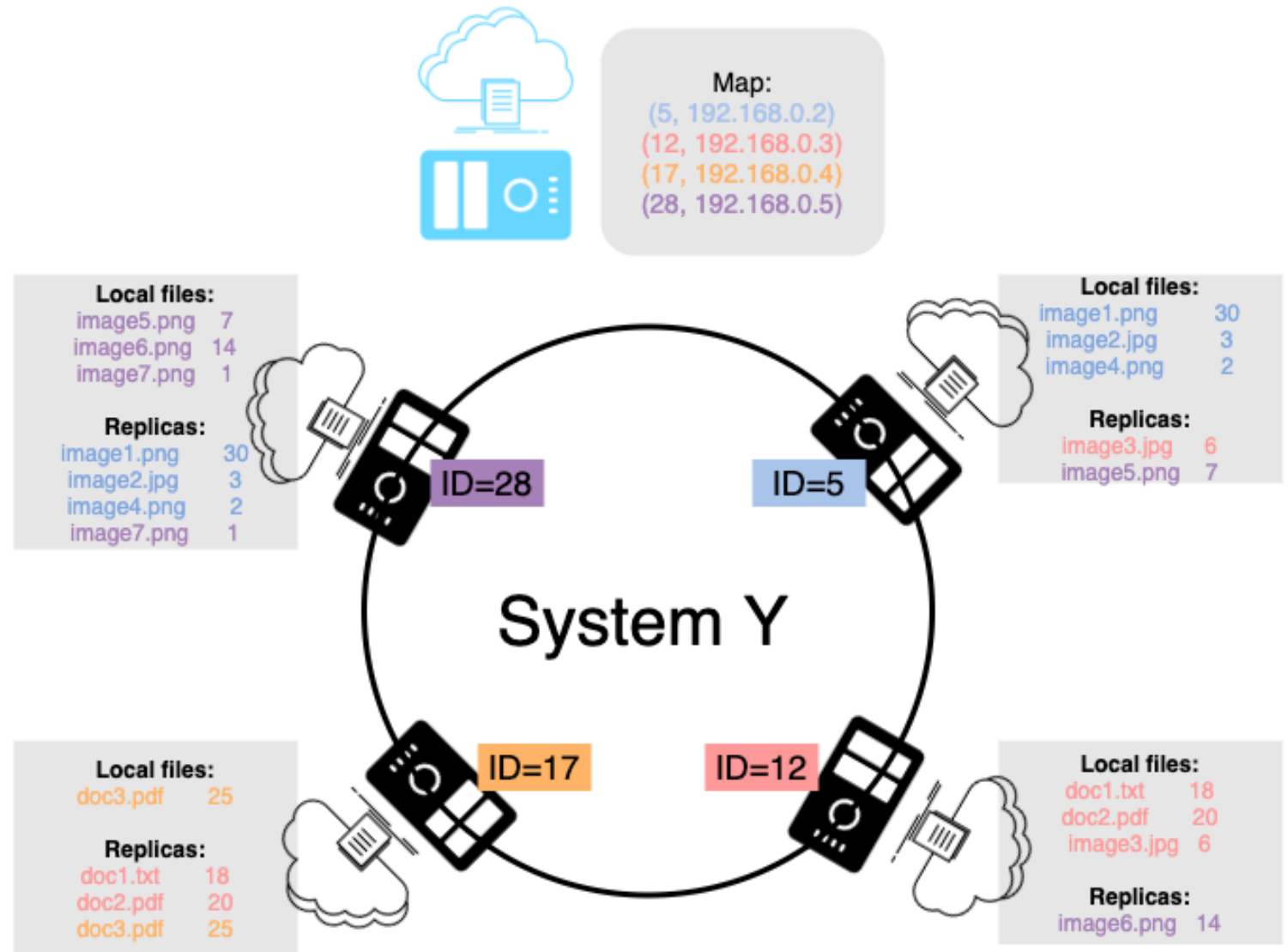
Naming server: specification 2, Example



Naming server: specification 2, Example



System Y example:



Tasks

- **Tests that should be enabled by Naming server:**
 - Add a node with a unique node name
 - Add a node with an existing node name
 - Send a filename and the IP address
 - Send a filename with a hash smaller than the smallest hash of the nodes
 - Send a filename and at the same time remove the node
 - Ask from two PCs for an IP address of a filename

Additional information

■ Hashing

- The standard hashing function in Java returns a value between $-2\,147\,483\,647$ and $2\,147\,483\,647$ since these values are too big for our project, we need to set new bounds for values between 0 and 32 768
- Mapping the default interval $(-2\,147\,483\,647, 2\,147\,483\,647)$ to the required interval $(0, 32\,768)$ is given below:
 - If $\text{max} = 2\,147\,483\,647$ and $\text{min} = -2\,147\,483\,647$, then `hashCode()` value can be calculated as:
 - $\text{hash} = (\text{hostname.hashCode()} + \text{max}) * (32768 / \text{max} + \text{abs}(\text{min}))$
 - This way, if `hostname.hashCode()` or `filename.hashCode()` returns $2\,147\,483\,647$, the result is $\text{hash} = 32768$, which is indeed a new maximum, and if `hostname/filename.hashCode()` returns $-2\,147\,483\,647$, hash will be 0, which is a new minimum
 - Any other hashcode value from the range $(2\,147\,483\,647, -2\,147\,483\,647)$, after this mapping denoted by hash , will be in the range $(0, 32768)$

Additional information

■ Map

- A Map is an interface in Java. A Map consists of (key, value) couples where key is unique in the Map. To get a value from the map, you must use the key. The Map interface gives a set of functions with WHAT can be done with the data, not HOW it's done. This is implemented in the classes which uses the interface (HashMap, TreeMap, EnumMap, ...)

Deadline

- **3 sessions**
 - Deadline report naming server -> 19/04/2025